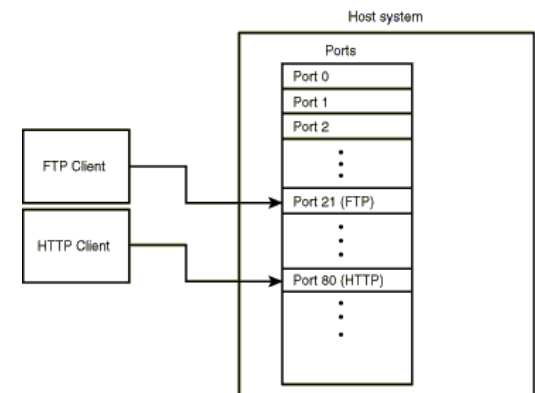


Računarske mreže

Soketi

Identifikacija usluge

- IP adresa jedinstveno određuje računar na Internetu, ali svaki računar može biti server većeg broja usluga (servisa)
- Na osnovu porta protokoli transportnog nivoa prepoznaju aplikaciju kojoj treba proslediti podatke
- Usluge se identifikuju brojem porta
- Port je 16-to bitni celi broj dodeljen nekom servisu
- Portovi do broja 1024 rezervisani su za standardne usluge



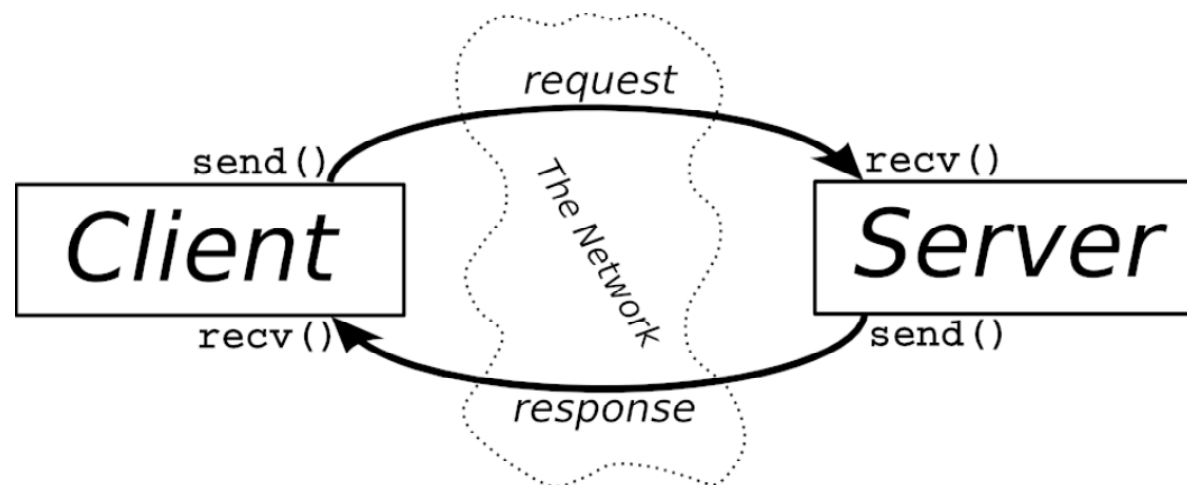
Podela portova

- Dobro poznati:
1 – 1023
- Registrovani:
1024 – 49151
- Dinamički (privatni):
49152 – 65535

Port	Protokol	Servis
20	TCP	FTP data
21	TCP	FTP control (command)
25	TCP	SMTP
53	TCP/UDP	DNS
67	UDP	DHCP server
68	UDP	DHCP client
80	TCP	HTTP
110	TCP	POP3
520	UDP	RIP

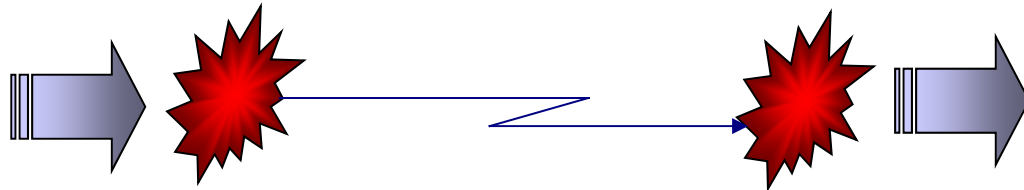
Klijent-server arhitektura

- Najveći deo saobraćaja na Internetu predstavlja uzajamnu komunikaciju parova mašina
- Da bi komunikacija bila moguća, mašine se moraju međusobno pronaći
- Mašina za koju se tačno zna “gde je” i koja osluškuje pozive naziva se **server**
- Mašina koja inicira komunikaciju naziva se **klijent**
- Server obično skladišti određene podatke i nudi usluge klijentu, a klijent je korisnih tih podataka i usluga

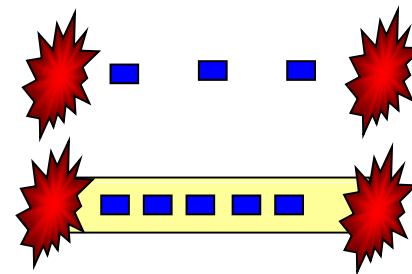


Soketi

- Soketi predstavljaju programsku apstrakciju za komunikacioni kanal, koji se uspostavlja između dva programa, tačnije između dva porta.



- Postoji tri tipa soketa:
 - datagram sockets
 - stream sockets
 - row sockets



1. Inicijalizacija Winsock-a

- Uključivanje zaglavlja i biblioteka:
`#include <winsock.h>`
`#pragma comment(lib, "wsock32.lib")`
- Inicijalizacija WSA strukture i aktiviranje WS2_32.DLL-a
`WSAData wsa;`
`WSAStartup(0x0101, &wsa); // Verzija 1`
`WSAStartup(0x0202, &wsa); // Verzija 2`

2. Kreiranje serverskog soketa

- Alokacija soketa

```
sockaddr_in sin;  
SOCKET sock;  
if((sock = socket(AF_INET, SOCK_STREAM, 0)) == SOCKET_ERROR)  
    return FALSE;
```

- Povezivanje sa portom i adresom

```
sin.sin_family = AF_INET; /* Postaviti adresnu familiju (način zadavanja adrese) */  
sin.sin_port = htons (m_nListenPort); /* Broj porta koji se osluškuje, npr.1234 */  
sin.sin_addr.s_addr = htonl (INADDR_ANY); /* 0.0.0.0 */  
if( bind(sock, (sockaddr *)&sin, sizeof(sockaddr_in)) == SOCKET_ERROR)  
    return FALSE;
```

- Započeti osluškivanje na datom portu

```
if( listen(sock, 100) == SOCKET_ERROR) return FALSE;
```

Socket

Funkcija **socket** kreira odgovarajući soket.

SOCKET socket (int *af*, int *type*, int *protocol*);

Parametri:

- *af*
 - Definiše adresnu familiju. U našem slučaju uvek će biti **AF_INET**
- *type*
 - Tip soketa. U verziji Windows Socket-a 1.1, podržana su samo dva, i to **SOCK_STREAM** i **SOCK_DGRAM**. U verziji 2 dodati su mnogi novi tipovi i ne moraju se definisati u ovom trenutku, jer aplikacija može dinamički odrediti attribute dostupnih protokola transportnog nivoa pomoću funkcije **WSAEnumProtocols**. Definicije tipova soketa mogu se naći u **WINSOCK2.H** datoteci.
- *protocol*
 - Protokol koji će biti korišćen u komunikaciji, npr **IPPROTO_TCP**. Ako se ne navede, tj. ako se prosledi 0, protokol se bira na osnovu prethodnog parametra (tipa soketa).

SOCKADDR_IN structura

```
struct sockaddr_in {  
    short            sin_family;  
    unsigned short   sin_port;  
    struct in_addr   sin_addr;  
    char             sin_zero[8];  
};  
  
struct in_addr {  
    union {  
        struct { u_char s_b1,s_b2,s_b3,s_b4; } S_un_b;  
        struct { u_short s_w1,s_w2; } S_un_w;  
        u_long S_addr;  
    } S_un;  
};
```

U Internet address familiji (AF_INET), **SOCKADDR_IN** struktura se koristi da specificira lokalnu ili udaljenu tačku u komunikaciji.

Komponente:

- **sin_family**
 - mora biti AF_INET
- **sin_port**
 - IP port
- **sin_addr**
 - IP adresa.
- **sin_zero**
 - dopuna da bi struktura bila iste veličine kao **SOCKADDR**.

Konverzija adresa i portova

Različiti procesori različito predstavljaju podatke. Neki u *big endian*, a neki u *little endian* formatu. Da bi se razumeli, formati zapisa paketa koji putuju mrežom moraju biti uniformni. Izabran je *big endian* zapis. Obzirom da Intel pamti podatke u *little endian* formatu, neophodna je konverzija. Za to postoje sledeće funkcije:

- **htons** – konvertuje **short** (16-bit) iz host zapisa (*little endian*) u network zapis (*big endian*)
- **htonl** – konvertuje **long** (32-bit) iz host zapisa u network zapis
- **ntohs** – konvertuje **short** (16-bit) iz network zapisa u host zapis
- **ntohl** – konvertuje **long** (32-bit) iz network zapisa u host zapis

Bind

Funkcija **bind** povezuje soket sa lokalnom komunikacionom tačkom (adresom i portom).

```
int bind ( SOCKET s,  
          const struct sockaddr FAR* name,  
          int namelen );
```

- **Parametri:**

- *s*
 - soket koji treba povezati.
- *name*
 - SOCKADDR struktura koja definiše lokalnu komunikacionu tačku.
- *namelen*
 - Veličina prethodne strukture.

Listen

Funkcija **listen** aktivira osluškivanje dolazećih konekcija.

```
int listen ( SOCKET s, int backlog );
```

- **Parametri:**

- *s*
 - soket
- *backlog*
 - maksimalna dužina reda u koji se smestaju konekcije koje čekaju da budu prihvaćene. Ako se navede SOMAXCONN, postavlja se maksimalna vrednost.

2a. Kreiranje serverskog soketa

- Uključivanje neblokirajućeg moda

WSAAsyncSelect (sock, hWnd, WM_USER + 1, FD_ACCEPT);

- Poziva se samo jednom za svaki soket. Ako je potrebno uključiti više događaja, oni se kombinuju kao flegovi - FD_ACCEPT | FD_READ | FD_CLOSE
- Šalje poruku prozoru koji je specificiran u pozivu funkcija kada se desi neki od navedenih događaja i to:
 - u **wParam** se nalazi socket na kome se desio događaj
 - u **lParam** se nalazi **događaj u nižem** delu i **greška u višem** delu (lParam je DWORD i višu i nižu reč treba tumačiti zasebno)

WSAAsyncSelect

Funkcija **WSAAsyncSelect** uključuje asinhronu notifikaciju događaja vezanih za soket.

```
int WSAAsyncSelect ( SOCKET s, HWND hWnd,  
                    unsigned int wMsg, long lEvent );
```

- **Parametri**

- *s*
 - Soket
- *hWnd*
 - *Handle* prozora koji će primiti poruku kada se specificirani događaj desi
- *wMsg*
 - Poruka koja se prima kada se događaj desi.
- *lEvent*
 - Flegovi koji definišu za koje mrežne događaje je aplikacija “zainteresovana”.

Inicijalizacija
serverskog sream
soketa – primer
potpune
implementacije

SOCKET listensock;
sockaddr_in sin;

```
bool CChatServerDlg::CreateServerSocket (UINT port)
{
    WSAData wsa;
    int error = WSAStartup (0x0202, &wsa); // Verzija 2
    if (error)    return false;
    if (wsa.wVersion != 0x0202) // da li je prava verzija Winsock-a
    {
        WSACleanup(); //Clean up Winsock
        return false;
    }
    if((listensock = socket (AF_INET, SOCK_STREAM, 0)) == SOCKET_ERROR)
    {
        AfxMessageBox("Socket error - create!");
        return false;
    }
    sin.sin_family = AF_INET;
    sin.sin_port = htons (port);
    sin.sin_addr.s_addr = htonl (INADDR_ANY);
    if( bind (listensock, (sockaddr *)&sin, sizeof(sockaddr_in)) == SOCKET_ERROR)
    {
        int Error = WSAGetLastError();
        AfxMessageBox("Socket error - bind!");
        return false;
    }
    if( listen (listensock, 100) == SOCKET_ERROR)
    {
        AfxMessageBox("Socket error - listen!");
        return false;
    }
    WSAAsyncSelect (listensock, this->m_hWnd, WM_USER + 1, FD_ACCEPT);
    return true;
}
```

3. Kreiranje klijentskog soketa

- **Kreiranje soketa**

```
SOCKET sock;
```

```
if((sock = socket(AF_INET, SOCK_STREAM, 0)) == SOCKET_ERROR)
```

```
    return false; // Greška
```


3a. Kreiranje klijentskog soketa

- Inicijalizacija konekcije

```
sockaddr_in RemoteSin;
```

```
RemoteSin.sin_family = AF_INET; /* Familija adresa je AF_INET */
```

```
RemoteSin.sin_port = htons(nPort); /* Odredišni (serverski) port */
```

```
if((RemoteSin.sin_addr.S_un.S_addr = inet_addr (sServerIP)) == INADDR_NONE)  
    return false; // Pogrešno postavljena adresa
```

```
// A može i kraće (umesto addr.S_un.S_addr samo s_addr)
```

```
if((RemoteSin.sin_addr.s_addr = inet_addr(sServerIP)) == INADDR_NONE)
```

```
    if( connect (sock, (sockaddr *)&RemoteSin, sizeof(sockaddr_in)) ==  
        SOCKET_ERROR) return false;
```

- Prelazak na asinhroni mod

```
WSAAsyncSelect (sock, m_hWnd, WM_USER + 2, FD_READ | FD_CLOSE);
```

Funkcija `inet_addr()`

Funkcija **`inet_addr`** konvertuje string koji sadrži IPv4 adresu u “tačkastoj” notaciji u odgovarajuću `IN_ADDR` strukturu

`unsigned long inet_addr ( const char FAR * cp );`

- Parametri:
 - `cp` – IPv4 adresa u “.” notaciji

Connect

Funkcija **connect** uspostavlja vezu datog soketa sa udaljenom tačkom koja se specificira sockaddr strukturom.

```
int connect ( SOCKET s,  
             const struct sockaddr FAR* name,  
             int namelen );
```

- **Parametri:**

- *s*
 - Soket
- *name*
 - Udaljena komunikaciona tačka
- *namelen*
 - Dužina prethodnog parametra.

4. Zatvaranje soketa i aplikacije

- Zatvaranje soketa
`closesocket(sock);`
- Zatvaranje Winsock-a (prilikom gašenja aplikacije treba počistiti za sobom)
`WSACleanup();`

5. Slanje podataka

- Primer slanja stringa
`send(sock, "Test String", 12, 0);`
- Primer slanja strukture
`MyStruct a;`
`send(sock, (char *)&a, sizeof(MyStruct), 0);`
- A možemo napisati i svoju funkciju za slanje stringova
`SendData(sock, "Ovo su podaci za slanje!");`
...
`int SendData(SOCKET sock, char * lpszData)`
`{`
 `return send(sock, lpszData, strlen(lpszData), 0);`
`}`

Send

Funkcija **send** šalje podatke kroz povezani *stream* soket.

```
int send ( SOCKET s, const char FAR * buf, int len, int flags );
```

- **Parametri:**

- *s*
 - Povezani soket.
- *buf*
 - Bafer koji sadrži podatke koje treba preneti.
- *len*
 - Dužina podataka u baferu.
- *flags*
 - Indikator koji definiše način slanja (MSG_DONTROUTE – ne rutirati, MSG_OOB – out-of-band slanje).

Sendto

Funkcija **sendto** šalje *datagram* na dato odredište.

```
int sendto ( SOCKET s, const char FAR * buf, int len,  
            int flags, const struct sockaddr FAR * to, int tolen );
```

- **Parametri:**

- *s*
 - Soket
- *buf*
 - Bafer koji sadrži podatke koje treba poslati.
- *len*
 - Količina podataka u baferu *buf*.
- *flags*
 - Indikatori
- *to*
 - Pointer na strukturu koja sadrži odredišnu adresu.
- *tolen*
 - Veličina adresne strukture.

6. Prijem podataka

- U okviru **WindowProc** funkcije treba dodati poziv koji će obratiti pristiglu poruku sa soketa

```
switch(uMsg)
{
...
case WM_USER + 2:
    HandleData(wParam, lParam);
    break;
...
}
```


6a. Prijem podataka

```
int HandleData(WPARAM wParam, LPARAM lParam)
{
    SOCKET sock = (SOCKET)wParam;
    WORD event = LOWORD(lParam);
    WORD error = HIWORD(lParam);
    if ( event == FD_CLOSE )
    {
        closesocket (sock);
    }
    else if ( event == FD_READ )
    {
        char szBuffer[1024];
        ZeroMemory(szBuffer, 1024);
        recv(sock, szBuffer, 1024, 0);
        CString str(szBuffer);
        AfxMessageBox(str);
    }
    return TRUE;
}
```

Recv

Funkcija **recv** prima podatke od povezanog *stream* soketa.

```
int recv ( SOCKET s, char FAR* buf, int len, int flags );
```

- **Parametri:**

- *s*
 - Soket
- *buf*
 - Bafer za prijem podataka.
- *len*
 - Veličina bafera.
- *flags*
 - Indikatori (MSG_PEEK – kopira podatke ali ih ne uklanja iz ulaznog reda čekanja, MSG_OOB – prihvatanje out-of-band podataka)

Recvfrom

Funkcija **recvfrom** prima datagram i odgovarajuću izvornu adresu i port

```
int recvfrom ( SOCKET s, char FAR* buf, int len, int flags,  
              struct sockaddr FAR* from, int FAR* fromlen );
```

- **Parametri:**
 - *s*
 - Soket
 - *buf*
 - Bafer za prijem podataka.
 - *len*
 - Veličina *buf* bafera.
 - *flags*
 - Indikatori
 - *from*
 - Bafer za prijem izvorne adrese i porta.
 - *fromlen*
 - Veličina *from* bafera.

Klijent – kompletni kod

```
bool CChatClientDlg::CreateClientSocket()
{
    WSADATA wsa;
    int error = WSAStartup(0x0202, &wsa); // Verzija 2
    if (error) return false;
    if (wsa.wVersion != 0x0202) {
        WSACleanup(); //Clean up Winsock
        return false;
    }
    RemoteSin.sin_family = AF_INET;
    RemoteSin.sin_port = htons(m_nServerPort);
    if((RemoteSin.sin_addr.s_addr = inet_addr(m_sServerIP.GetBuffer
        (m_sServerIP.GetLength())) == INADDR_NONE) return false;
    if((sock = socket(AF_INET, SOCK_STREAM, 0)) == SOCKET_ERROR)
        return false;
    if( connect (sock, (sockaddr *)&RemoteSin, sizeof(sockaddr_in)) == SOCKET_ERROR)
        return false;
    WSAAsyncSelect (sock, this->m_hWnd, WM_USER + 2, FD_READ | FD_CLOSE);
    return true;
}
```

```
CString m_sServerIP;
UINT m_nServerPort;
SOCKET sock;
sockaddr_in RemoteSin;
```

Klijent – kompletni kod (nastavak)

```
void CChatClientDlg::OnExit()
{
    closesocket(sock);
    WSACleanup();
    exit(0);
}

int CChatClientDlg::SendData(SOCKET sock, char * lpszData)
{
    return send (sock, lpszData, strlen(lpszData), 0);
}

LRESULT CChatClientDlg::WindowProc(UINT message, WPARAM wParam, LPARAM lParam)
{
    if(message == WM_USER+2)
    {
        HandleData(wParam, lParam);
    }
    return CDialog::WindowProc(message, wParam, lParam);
}
```

Klijent – kompletni kod (nastavak)

```
int CChatClientDlg::HandleData(WPARAM wParam, LPARAM lParam) {
    SOCKET sock = (SOCKET)wParam;
    WORD event = LOWORD(lParam);
    WORD error = HIWORD(lParam);
    if( event == FD_CLOSE ) {
        closesocket(sock);
    }
    else if( event == FD_READ ) {
        char szBuffer[1024];
        ZeroMemory(szBuffer, 1024);
        recv(sock, szBuffer, 1024, 0);
        CString str(szBuffer);
        // ...
    }
    return TRUE;
}

void CChatClientDlg::OnButtonSend(){
    UpdateData(TRUE);
    // ...
    SendData(sock, m_sInputString.GetBuffer(m_sInputString.GetLength()));
    // ...
}
```

7. Prihvatanje konekcije

- Kada pristigne konekcija, server treba da je prihvati. To se dešava u **WinProc** funkciji, kada naiđe odgovarajuća poruka...

```
switch(uMsg)
{
...
case WM_USER + 1:
    HandleAccept(wParam, lParam);
    break;
...
}
```

7a. Prihvatanje konekcije

```
int HandleAccept(WPARAM wParam, LPARAM lParam)
{
    SOCKET sock = (SOCKET)wParam;
    WORD event = LOWORD(lParam);
    WORD error = HIWORD(lParam);
    if( event == FD_ACCEPT )
    {
        sockaddr_in NewRemoteSin;
        SOCKET newsock = accept (sock, (sockaddr*)&NewRemoteSin, NULL);
        /* Za komunikaciju se kreira novi soket. NewRemoteSin sadrži adresu klijenta. */
        WSAAsyncSelect(newsock, hWnd, WM_USER + 2, FD_READ | FD_CLOSE);
        SendData(newsock, "Dobro dosao na moj server!");
    }
    return TRUE;
}
```


Accept

Funkcija **accept** prihvata konekciju na serverskoj strani i kreira novi soket za komunikaciju sa klijentom.

SOCKET accept (SOCKET s, struct sockaddr FAR* addr, int FAR* addrlen);

- **Parametri:**

- *s*
 - *Listening* (serverski) soket koji je prihvata konekciju. Funkcija kreira novi soket preko koga se zapravo ostvaruje komunikacija sa klijentom. Serverski soket nastavlja da osluškuje nove konekcije.
- *addr*
 - Pokazivač na strukturu u koju se smeštaju podaci o klijentu.
- *addrlen*
 - Veličina prethodne strukture.

Server – kompletan kod

```
LRESULT CChatServerDlg::WindowProc(UINT message,
                                   WPARAM wParam, LPARAM lParam)
{
    if(message == WM_USER+1)
    {
        HandleAccept(wParam, lParam);
    }
    else if(message == WM_USER+2)
    {
        HandleData(wParam, lParam);
    }
    return CDialog::WindowProc(message, wParam, lParam);
}
```

Server – kompletan kod (nastavak)

```
bool CChatServerDlg::HandleAccept(WPARAM wParam, LPARAM lParam)
{
    SOCKET sock = (SOCKET)wParam;
    WORD event = LOWORD(lParam);
    WORD error = HIWORD(lParam);
    if(event == FD_ACCEPT)
    {
        //...
        {
            newsock[m_no] = accept (sock, (sockaddr*)&NewRemoteSin, NULL);
            WSAAsyncSelect(newsock[m_no], this->m_hWnd, WM_USER + 2,
                          FD_READ | FD_CLOSE);

            m_no++;
        }
    }
    return TRUE;
}
```

Očitavanje grešaka

- Kada se desi greška, funkcija obično vraća `SOCKET_ERROR` i postavlja kod greške. Kod greške može se preuzeti funkcijom `WSAGetLastError()`

```
int Error = WSAGetLastError();  
if (Error == WSANOTINITIALISED)  
...  
...
```

Primeri kodova:

WSAEACCES (10013) - *Permission denied.*

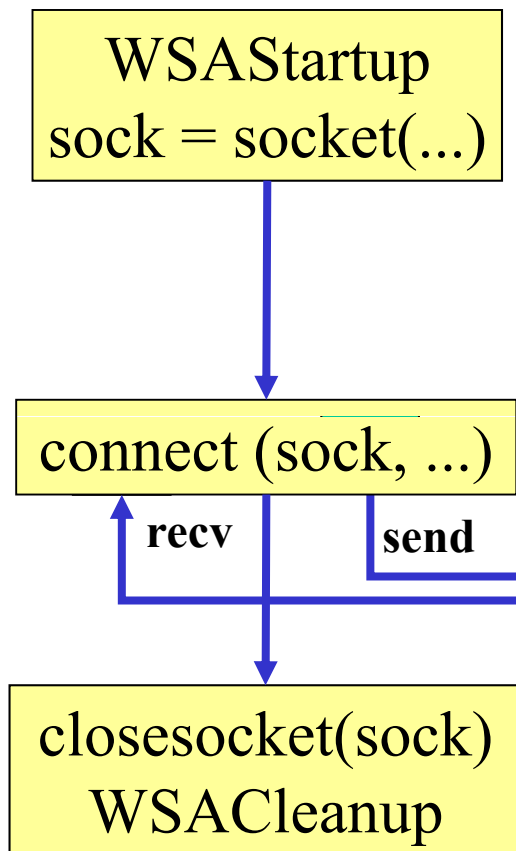
WSAEADDRINUSE (10048) - *Address already in use.*

WSAEADDRNOTAVAIL (10049) - *Cannot assign requested address.*

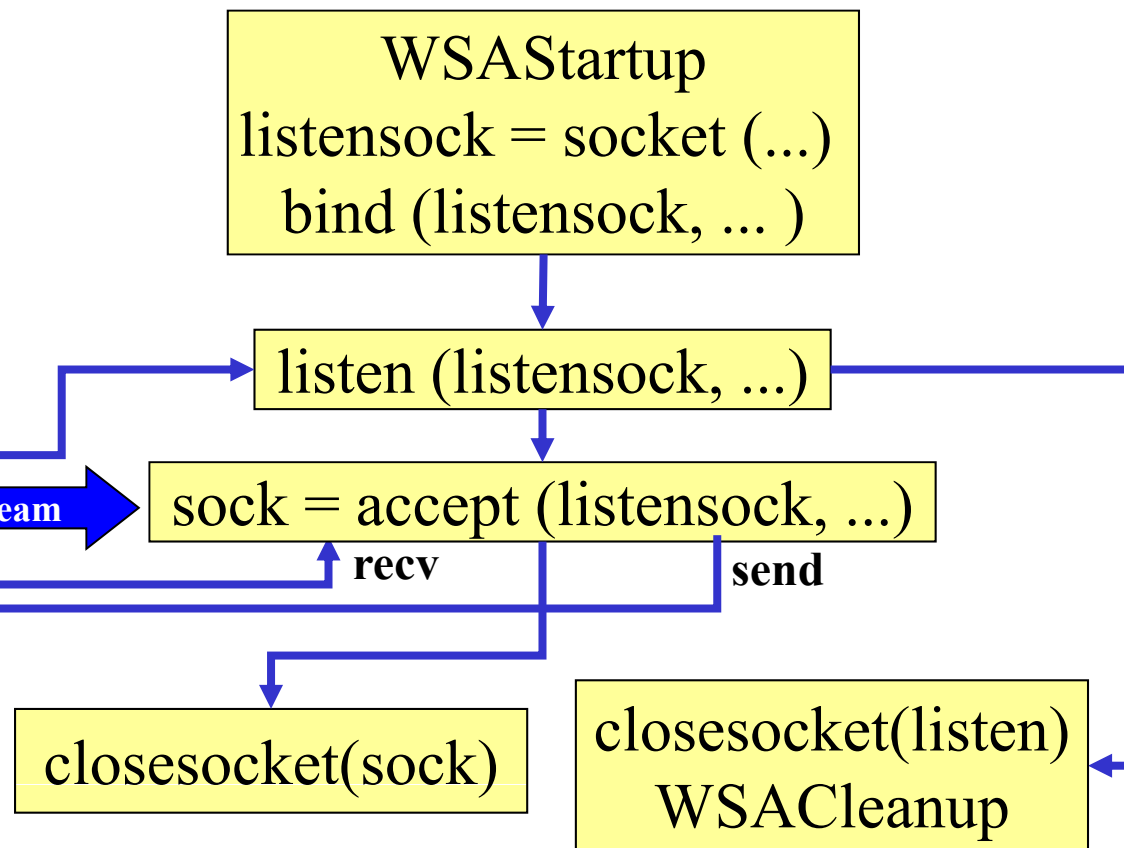
...

Pregled rada *stream* soketa

Klijent



Server



Datagram (UDP) server

```
SOCKET sock;  
int addr_len, bytes_read;  
char recv_data[1024];  
char send_data[1024];  
sockaddr_in server_addr, client_addr;  
if ((sock = socket (AF_INET, SOCK_DGRAM, 0)) == SOCKET_ERROR)  
{  
    exit(1);  
}  
  
server_addr.sin_family = AF_INET;  
server_addr.sin_port = htons(5000);  
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);  
if (bind (sock, (sockaddr *)&server_addr, sizeof(sockaddr_in))  
    == SOCKET_ERROR) {  
    exit(1);  
}
```

Datagram (UDP) server - nastavak

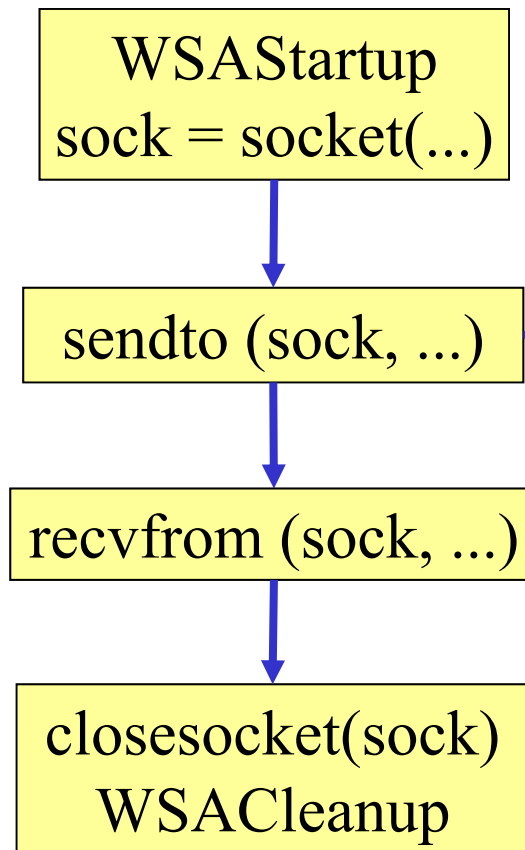
```
addr_len = sizeof(sockaddr);  
while (true)  
{  
    bytes_read = recvfrom(sock, recv_data, 1024, 0,  
                           (sockaddr *)&client_addr, &addr_len);  
  
    //... Neka obrada zahteva  
  
    sendto(sock, send_data, strlen(send_data), 0,  
           (sockaddr *)& client_addr, sizeof(sockaddr));  
}
```

Datagram (UDP) klijent

```
SOCKET sock;  
sockaddr_in server_addr;  
char recv_data[1024];  
char send_data[1024];  
if ((sock = socket (AF_INET, SOCK_DGRAM, 0)) == SOCKET_ERROR){  
    exit(1);  
}  
server_addr.sin_family = AF_INET;  
server_addr.sin_port = htons(5000);  
server_addr.sin_addr.s_addr = inet_addr(m_sServerIP);  
while (true) {  
    // Slanje zahteva  
    sendto(sock, send_data, strlen(send_data), 0,  
           (sockaddr *)& server_addr, sizeof(sockaddr));  
    // Prijem rezultata obrade  
    int bytes_read = recvfrom(sock, recv_data, 1024, 0,  
                               (sockaddr *)&client_addr, &addr_len);  
}
```


Pregled rada *datagram* soketa

Klijent



Server

